INTERNSHIP TECHNICAL REPORT

+ DEVELOPMENT OF A PIC32-BASED MOTOR CONTROLLER

Done By

T.Sai Alekhya & Pathipati Meghana

Under The guidance of

D.V.S Phanindra

ABSTRACT

This project presents the development of a rudimentary closed-loop DC motor control system, designed for controlling the telescope motion using the PIC32MK MCM Curiosity Pro development board. The system implements a real-time PID control algorithm that dynamically adjusts motor speed based on feedback from an optical slot sensor. Motor actuation is achieved using pulse-width modulation (PWM) generated through the microcontroller's Output Compare (OC) module, and control signals are delivered via an opto-isolated driver circuit. UART-based communication enables real-time monitoring and interaction. The firmware was developed in Embedded C using the MPLAB X IDE. The project demonstrates development and integration of hardware and firmware to achieve real-time control of a DC motor integrated in the lab setup. Future enhancements that can be made towards developing a PIC32-based motor controller for a telescope system will be discussed.

CONTENTS

CHAPTER 1: INTRODUCTION 4
1.1 Introduction 4
1.2 Objectives 5
1.3 Problem Statement 5
1.4 Proposed Solution 6

CHAPTER2: TASKS PERFORMED 6
2.1 System block diagram 6
2.2 Hardware setup 7
2.3 Motor driver design 7
2.4 Closed-loop PID block diagram 9
2.5 Tachometer validation and curve fitting for feedforward model 10

CHAPTER3: FIRMWARE DEVELOPMENT 12
3.1 Functional Overview of Frimware 12
3.2 Module Initialization 133

CHAPTER 4: PID TUNING AND OBSERVATIONS 14
4.1 Tuning Method 14
4.2 Observations 14

CHAPTER 5: RESULTS 15
5.1 Measured speed in COM terminal when target speed is set to 900 RPM 15
5.2 Tachometer reading when target speed is set to 900 RPM 15
5.3 Speed regulation at various load conditions 15
5.4 Effect of PID control on motor speed across varying load when target speed is set to 1000 RPM 15

CHAPTER 6: CONCLUSION 15

CHAPTER 7: FUTURE WORK 15

CHAPTER 8: REFERENCES 15

CHAPTER 1: INTRODUCTION

1.1 Introduction

Embedded systems are specialized computing platforms designed to perform dedicated functions with constraints on real-time responsiveness, reliability, and power efficiency. These systems are widely applied in industrial automation, aerospace, medical instrumentation, and scientific research. One such area where embedded control plays a crucial role is in the automation of telescope motion systems, where continuous and accurate alignment with celestial objects is required due to the Earth's rotation.

This project focuses on the development of a rudimentary closed-loop DC motor control system for basic telescope motion control, implemented using the PIC32MK MCM Curiosity Pro development board. The aim is to design and test a functional embedded control system capable of regulating motor speed in real time, using sensor feedback and a control algorithm.

A Proportional-Integral-Derivative (PID) control algorithm is implemented in firmware to regulate the motor speed based on feedback from an optical slot sensor. The sensor detects shaft rotation by generating pulses, which are processed to calculate the motor's actual speed (in RPM). This measured RPM is compared with a predefined reference, and the controller updates the motor actuation signal accordingly to minimize the speed error.

Motor control is achieved using Pulse Width Modulation (PWM) generated through the Output Compare (OC) module of the PIC32MK microcontroller. The PWM output drives a custom MOSFET-based motor driver circuit, which includes 4N35 opto-isolators to ensure electrical isolation between the low-voltage microcontroller signals and the high-power motor circuit. This isolation enhances system safety and reduces electrical noise interference.

To enable real-time monitoring and interaction, UART-based serial communication is employed. System parameters such as actual RPM and applied PWM duty cycle are transmitted to a serial terminal, allowing live observation of motor behavior and easier debugging.

The firmware was developed in Embedded C using the MPLAB X IDE and compiled with the XC32 compiler. Basic tuning of PID parameters was carried out through experimental testing to achieve stable motor speed regulation under no-load conditions. A handheld digital tachometer was used to validate the RPM readings obtained from the slot sensor. Additionally, Python-based data analysis was performed to study the relationship between PWM duty cycle and RPM. This helped in understanding the system dynamics and provided initial data for feedforward control estimation.

Although the system is elementary and not intended for high-precision applications, it successfully demonstrates the following core embedded system concepts:

1. Implementation of a basic closed-loop motor control system
2. Real-time feedback acquisition and PWM-based actuation
3. Use of opto-isolated circuitry for safe and modular power interfacing
4. UART-based monitoring and interaction for debugging and validation
5. Application of experimental methods for controller tuning and system analysis

This project lays the foundation for more advanced embedded motion control systems that could be integrated into telescope platforms. Future improvements may include support for multi-axis control, enhanced sensor resolution, advanced control strategies, and software integration for celestial object tracking.

1.2 Objectives

- To develop a stable PI-based DC motor speed control system using PIC32MK1024MCM100.
- To implement a real-time feedback loop for dynamic motor control using a slot sensor.
- To build a custom motor driver circuit using opto-isolators and MOSFETs.
- To validate system accuracy using a digital tachometer.
- To perform curve fitting using Python for modeling motor behavior and improving control logic.

1.3 Problem Statement

Accurate tracking of celestial objects requires precise, real-time control of telescope motion. Open-loop or manual systems often introduce positional errors due to environmental disturbances (such as wind or vibration), motor backlash, and mechanical imperfections.

These challenges can lead to image blur, drift, and reduced observational accuracy especially in long-duration tracking. There is a need for an embedded feedback-based control system capable of dynamically adjusting motor operation to maintain consistent speed and position, even under external disturbances.

1.4 Proposed Solution

To address the above issues, a closed-loop feedback control system was implemented using the PIC32MK1024MCM100 microcontroller. A slot sensor was used to measure motor RPM, and a Proportional-Integral (PI) controller was implemented in firmware to minimize the difference between the desired and actual motor speeds.

Key features of the solution include:

- Real-time PWM generation using OC5 output compare module.
- RPM computation through pulse timing from the slot sensor.
- Data logging and real-time monitoring using UART6.
- Hardware control using a custom MOSFET driver circuit (IRFZ44N) with 4N35 opto-isolation.
- Curve fitting using Python (NumPy, Plot) to derive duty-RPM relationships for validation.

The resulting system delivers stable and automated DC motor control, with potential applications in telescope automation, instrumentation, and precision robotics.

CHAPTER2: TASKS PERFORMED

2.1 SYSTEM BLOCK DIAGRAM

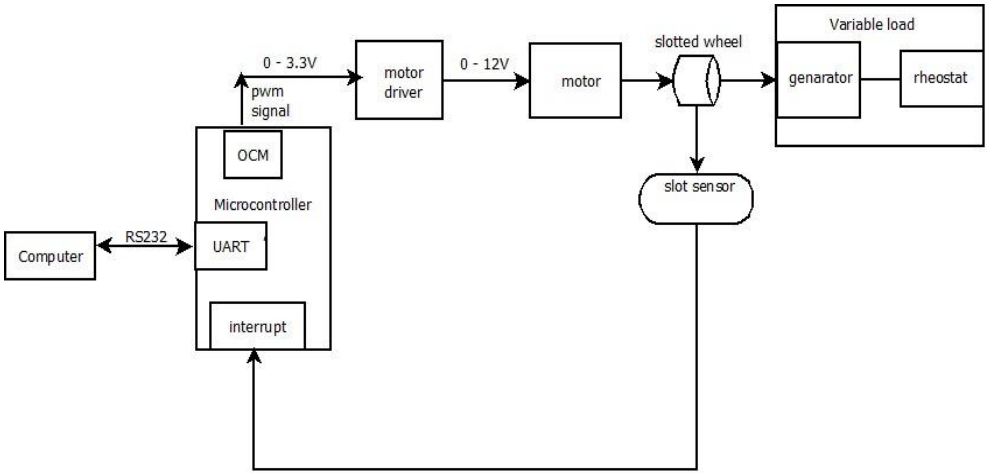


Fig1: SYSTEM BLOCK DIAGRAM

2.2 HARDWARE SETUP

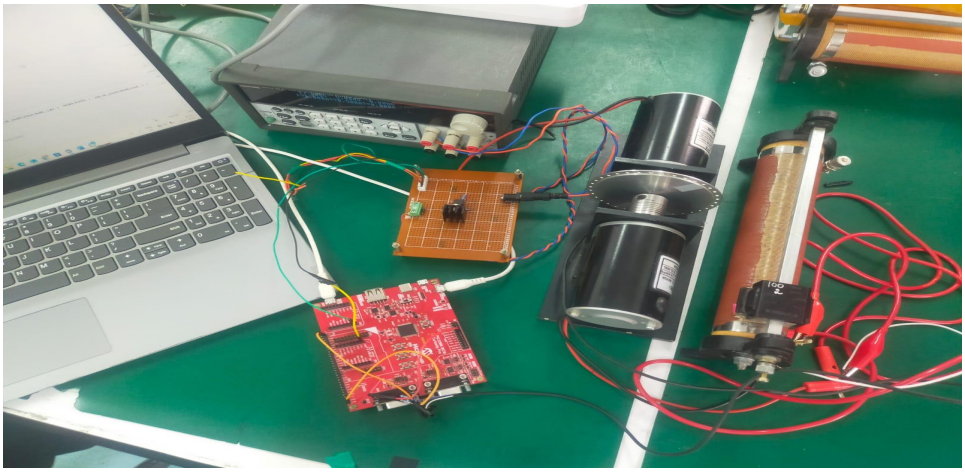


Fig 2: hardware setup

2.3 MOTOR DRIVER DESIGN

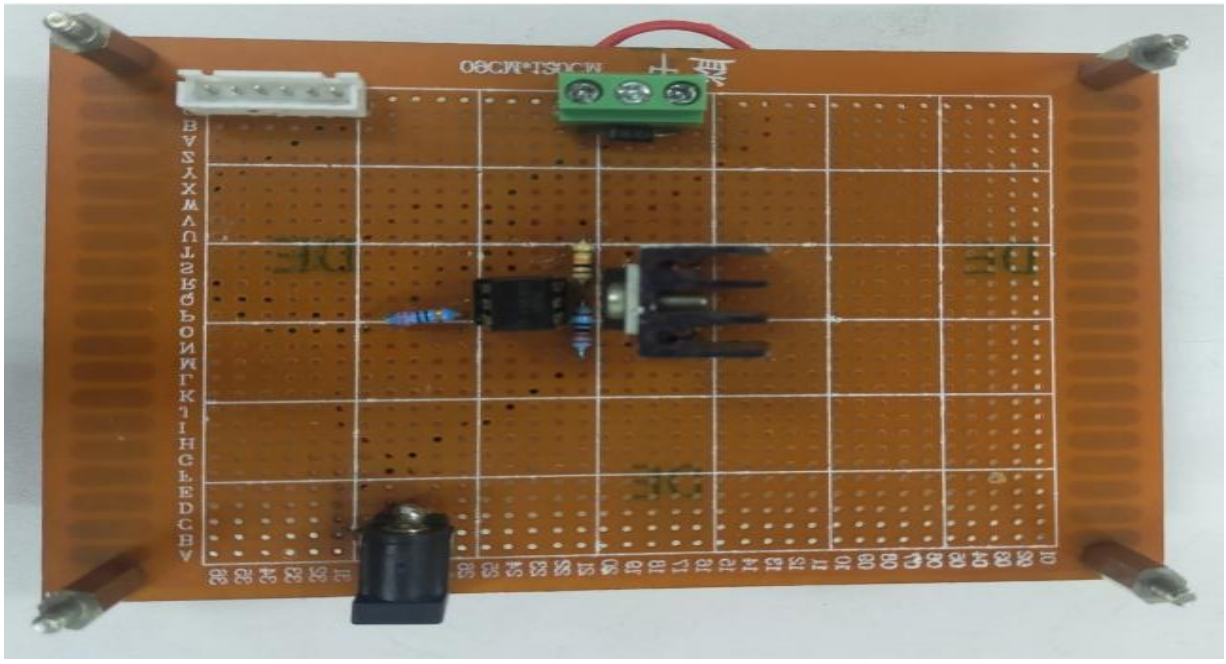


Figure 3: Motor Driver Hardware Circuit

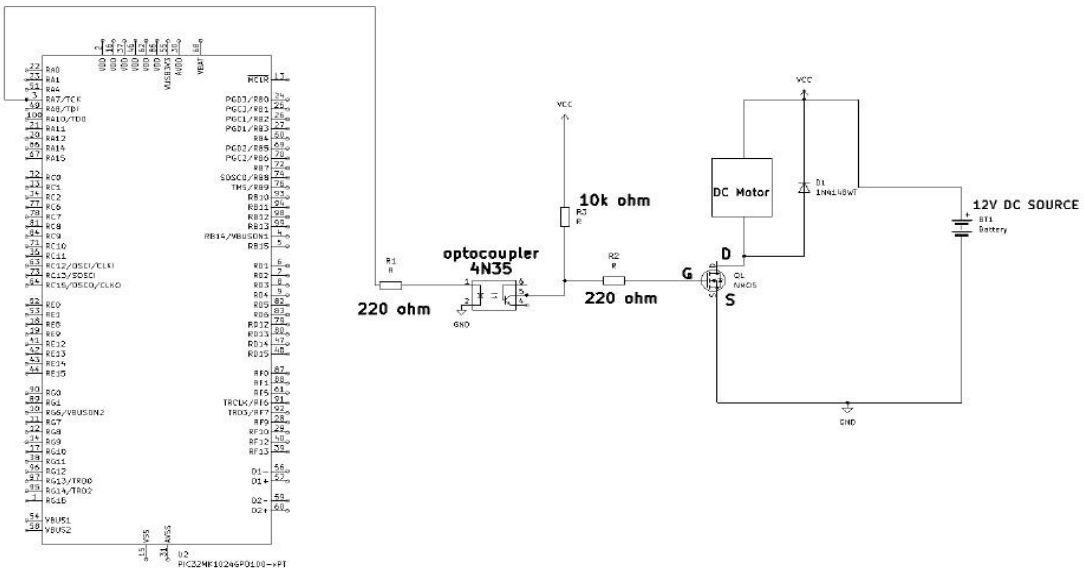


Figure 4: Motor Driver Block Diagram

Optocoupler (4N35):

- Electrically isolates the microcontroller (low-power side) from the motor driver (high-power side).
- Protects the controller from voltage spikes and back EMF generated by the motor.
- Prevents ground loop issues by separating control and power grounds.
- The LED side is driven by the microcontroller output pin (e.g., RA7) through a 220Ω resistor.

MOSFET (IRFZ44N):

- Acts as a high-speed power switch.
- The gate is controlled by the output of the optocoupler (via a 220Ω resistor).
- The drain is connected to one terminal of the DC motor; source is grounded.
- Allows efficient switching of the motor using the PWM signal.

2.4 CLOSED LOOP PID BLOCK DIAGRAM

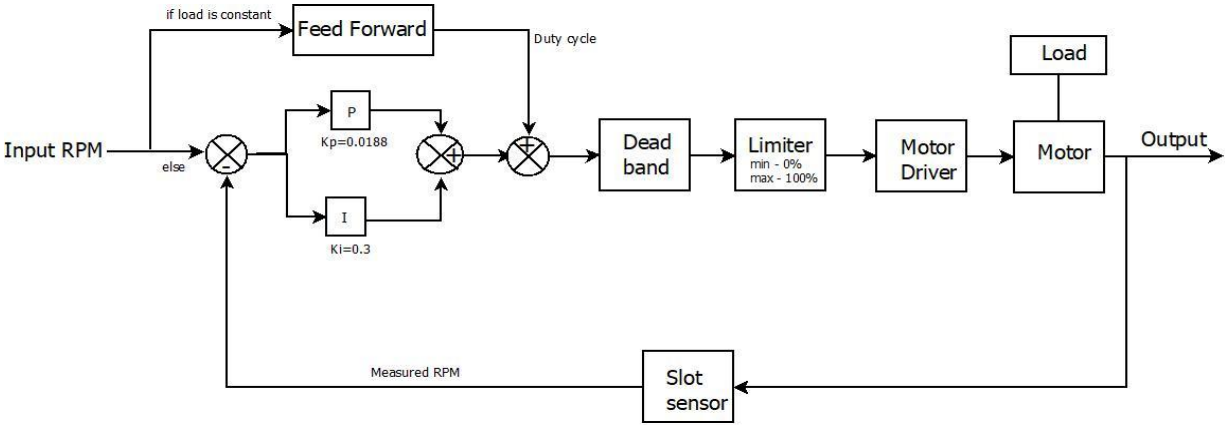


Fig5:PID Block Diagram

2.4.1 Block Description

Input RPM (Reference Input):
The desired rotational speed (setpoint) to be achieved by the motor, given in RPM.
Error Calculation:
The system calculates the difference between the desired input RPM and the measured RPM from the slot sensor.

e(t)=Input RPM−Measured RPM

PI Controller:
Proportional (P): Reacts to the current error with a gain of Kp = 0.0188 to correct deviations immediately.
Integral (I): Accumulates past errors using Ki = 0.3, ensuring elimination of steady-state error.
The PI output is a part of the control duty cycle.

Feed Forward Control:
Used when the load is constant.
Directly contributes to the duty cycle based on known system behavior, reducing the burden on the PI controller.
Helps in faster and more efficient control.
Summing Block:
Combines the outputs of PI and Feed Forward blocks to compute the total control signal (Duty Cycle).
Deadband:
Introduces a range around zero control input where no motor action is taken, preventing minor errors or noise from triggering unnecessary motion.
Improves stability and prevents motor jittering due to very small changes.
Limiter:
Restricts the duty cycle signal between 0% (minimum) and 100% (maximum).
Ensures the control output stays within hardware limits, preventing overdriving the motor.

Motor Driver:
Ampilfies the PWM signal and powers the motor accordingly. This can be implemented using opto-isolated MOSFETs as mentioned in your setup.
Motor:
Executes the mechanical action based on the received signal.
Drives the load, which could be a mechanical system like a telescope, robotic arm, etc.
Slot Sensor:
Provides real-time Measured RPM by generating pulses as the motor rotates.
Feedback is used in the control loop for continuous error correction.
Output:
Final RPM of the motor after control, fed back via the slot sensor.

2.5 TACHOMETER VALIDATION AND CURVE FITTING FOR FEEDFORWARD MODEL

To validate the accuracy of the RPM measurements obtained through the slot sensor and to analyze the behavior of the motor with respect to the PWM duty cycle, a handheld digital tachometer was used. The motor was operated at various duty cycles, and corresponding RPM values were manually recorded.

These readings were later used to create a polynomial fit between the duty cycle and RPM using Python. This graph and its fitted equation are useful in both validation and open-loop prediction of speed control behavior.

Recorded Data Table:-

Duty Cycle (%)	RPM
5	1280
10	1180
15	1125
20	980
25	930
30	780
35	730
40	550
45	520
50	360
55	330
60	180
65	120
70	0
80	0

Polynomial Fit Plot

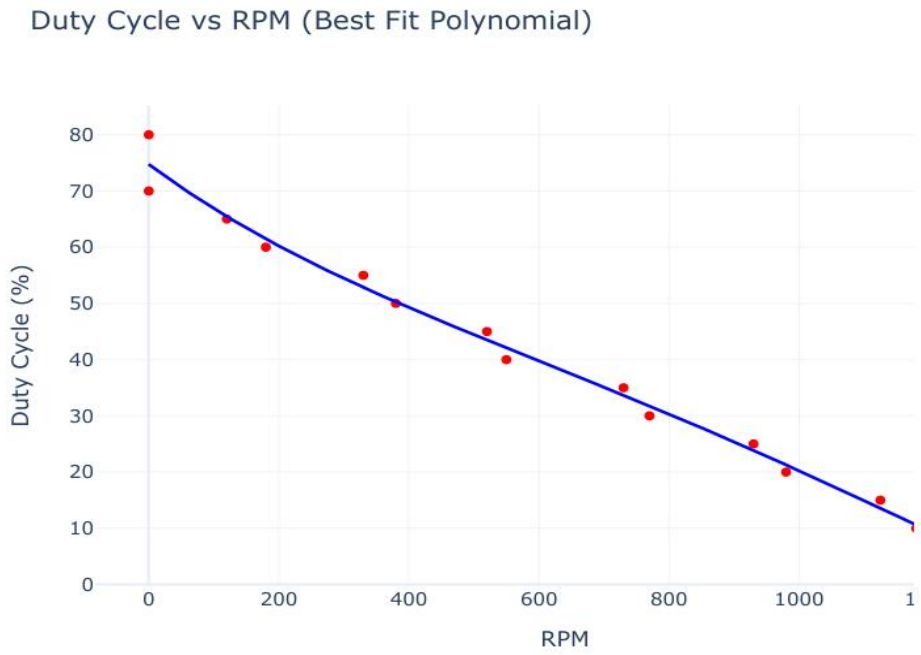


Figure 6: Duty Cycle vs RPM

Fitted Polynomial Equation

The best-fit polynomial equation (Duty as a function of RPM) is:

$$\text{Duty} = 7.8673\text{e-}05 \times x^2 - 8.5837\text{e-}02 \times x + 74.730$$

This equation helps predict the required duty cycle for a desired RPM and validates the response of the PI controller.

CHAPTER 3: FIRMWARE DEVELOPMENT

3.1 Functional Overview of Frimware

The firmware for the telescope motor control system was developed using a bare-metal programming approach on the PIC32MK1024MCM100 microcontroller. The development followed a modular and structured methodology, ensuring efficient initialization, real-time processing, and accurate motor control.

• The process began with system-wide initialization, including configuring the system clock, GPIO pins, UART6 communication, timers (Timer2 and Timer3), Output Compare (OC) module for PWM generation, and external interrupts (INT2 and INT3) for slot sensor

• feedback. Initialization was done manually through low-level register manipulation without using third-party libraries or frameworks.

• UART6 was configured to receive commands from a host system, enabling the motor to be started, stopped, or reversed. A circular command buffer was implemented using delimiters (STx and ETx) to detect and process commands efficiently within the UART6 receive ISR.

The external interrupt (INT3) was configured on the rising edge of the slot sensor signal to count slots and measure the time interval between them using Timer3. This time interval (dt) was used to calculate the motor's current speed (RPM).

• Timer2 was configured to generate periodic interrupts to execute a Proportional-Integral (PI) control loop. The control algorithm computes the error between the reference RPM and the measured RPM, updates the integral term, and calculates the control output. This output is then converted into a PWM duty cycle by updating the OC5RS register, which adjusts motor speed in real-time

. • Interrupts were prioritized and enabled at appropriate times based on motor direction to avoid conflicts and ensure correct slot counting. The firmware also includes diagnostic UART output for real-time feedback on RPM, reference speed, and control effort.

• The firmware structure separates initialization, command handling, ISR routines, and control logic, enabling readability, maintainability, and real-time performance critical for closed- loop motion control in astronomical tracking applications.

3.2 Module Initialization

• GPIO: LEDs configured as output (RG12–RG14), motor direction on RB14, slot sensor input on RD6.

• UART6: Configured at 115200 bps with RX on RA15, TX on RPA4 using PPS remapping.

• Timer2: Configured in 32-bit mode, 1:8 prescaler, PR2 = 37500 to generate 100 Hz interrupts for PI loop.

• Timer3: Free-running timer, 1:256 prescaler, used for measuring dt between slot pulses.
• OC5: PWM mode using Timer2, output on RA7, initialized to 50% duty cycle. Controlled by OC5RS.

• INT3: Rising edge interrupt on RD6 for slot pulse counting (slot sensor).

• Clock and EVIC: System clocks (unlocked and configured manually), peripheral modules enabled/disabled selectively, multi-vector interrupt control.

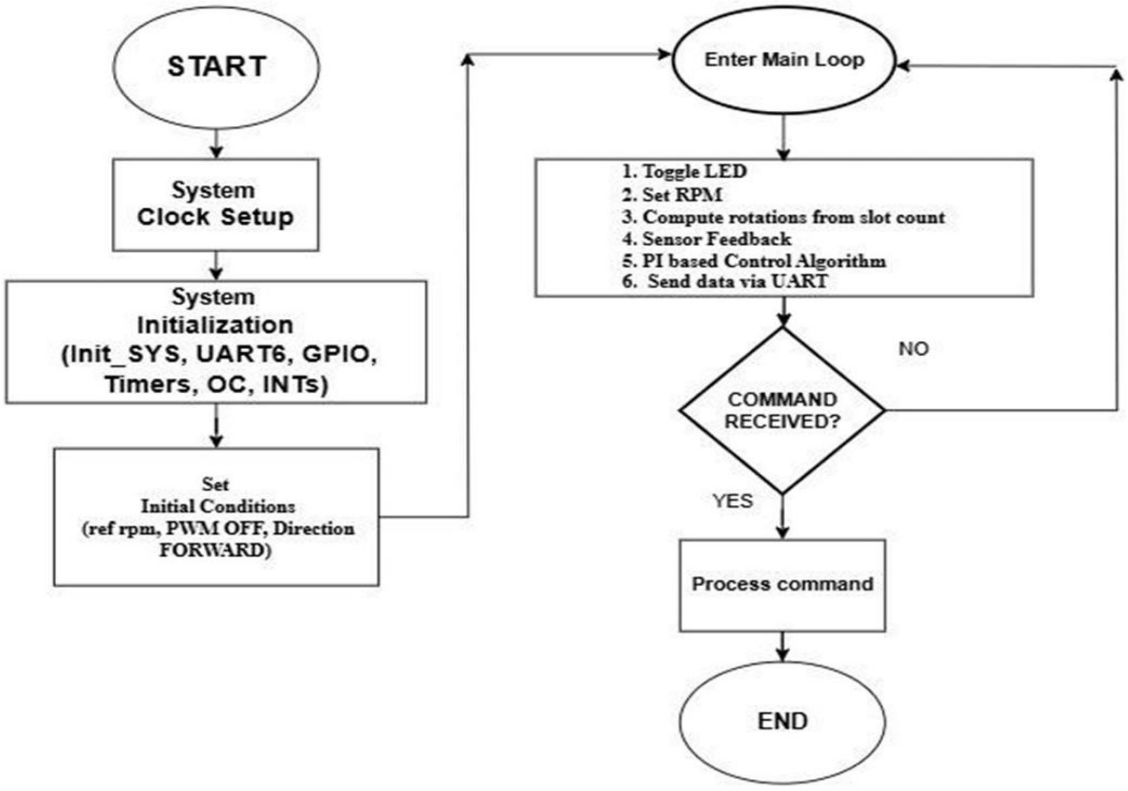


Fig7:Control Flow of Embedded Firmware

CHAPTER 4: PID TUNING AND OBSERVATIONS

The tuning of the PI controller was done experimentally by monitoring the measured RPM and PWM duty cycle values printed through the COM terminal. The goal was to minimize the difference between the target RPM and measured RPM using trial-and-error method.

4.1 Tuning Method

1. The initial values were set as: Kp = 0.01, Ki= 0.0
2. The target RPM was fixed (900 RPM), and the actual RPM printed on the COM terminal was observed.
3. Kp was gradually increased while watching how quickly the measured RPM approached the target value and how stable the PWM output remained.
4. Once a reasonable Kp was found, Ki was increased step-by-step to remove any steady deviation in the measured RPM.
5. The final tuned values were Kp = 0.0188 and Ki = 0.3

4.2 Observations

Condition	Observation
Low Kp	The motor did not rotate. It showed signs of trying to rotate (slight vibrations) but was unable to overcome static friction. The PWM duty cycle remained nearly constant, and there was no significant RPM change seen in the COM terminal.
High Kp	The motor produced audible noise or whining sounds. The RPM values fluctuated frequently, and the PWM duty cycle kept varying in the COM output. The system became sensitive to small errors, leading to unstable behavior.
Low Ki	The motor rotated and approached the target RPM, but the measured RPM stayed consistently below the reference value. The steady-state error remained and did not correct over time.
High Ki	The system became unstable. Both RPM and PWM duty cycle values printed in the COM terminal fluctuated continuously. The controller overreacted to small errors, causing oscillations.

4.3 Final Tuned Behavior

With **Kp = 0.0188** and **Ki= 0.3**, the system achieved stable speed regulation. The measured RPM closely tracked the reference RPM, and the PWM output adjusted smoothly with minimal fluctuation. This tuning provided a good balance between responsiveness and stability under no-load conditions.

CHAPTER 5: RESULTS

5.1 MEASURED SPEED IN COM TERMINAL WHEN TARGET SPEED IS SET TO 900 RPM

Transmit Data:

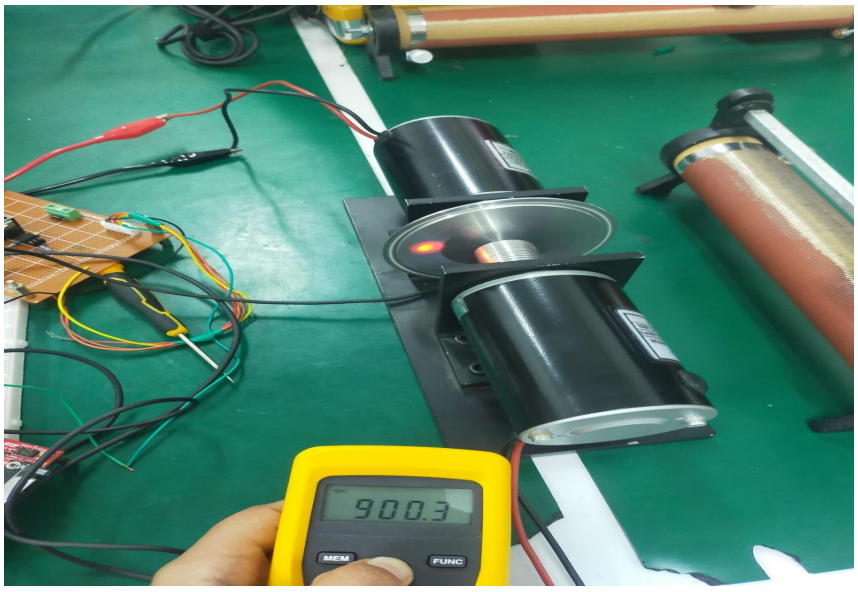
*r0900#

Send Data

Received Data:

#*R+00008633,+00900,+00900,+00000
#*R+00008633,+00900,+00900,+00000
#*R+00008633,+00900,+00900,+00000
#*R+00008633,+00900,+00900,+00000
#*R+00008589,+00900,+00897,+00003
#*R+00008606,+00900,+00900,+00000
#*R+00008562,+00900,+00897,+00003
#*R+00008477,+00900,+00893,+00007
#*R+00008546,+00900,+00902,-00002
#*R+00008490,+00900,+00897,+00003
#*R+00008537,+00900,+00902,-00002
#*R+00008423,+00900,+00893,+00007
#*R+00008462,+00900,+00900,+00000
#*R+00008462,+00900,+00900,+00000

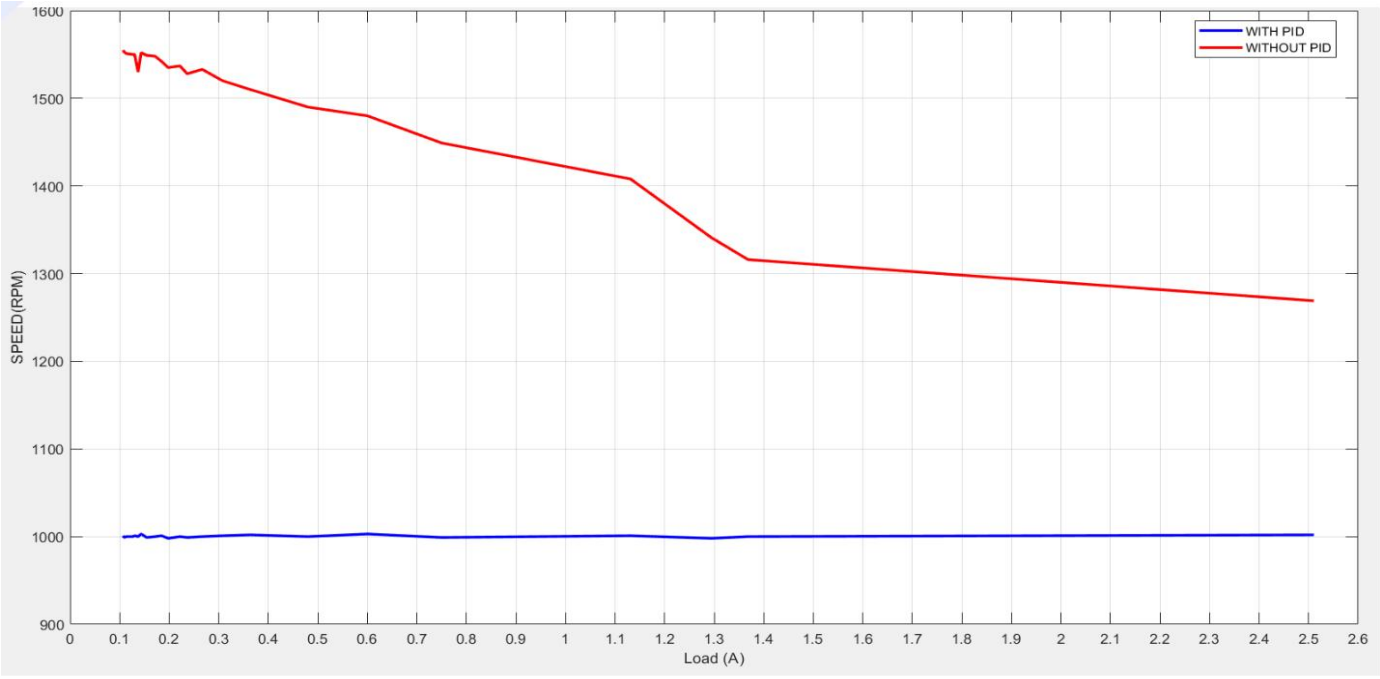
5.2 TACHOMETER READING WHEN TARGET SPEED IS SET AS 900 RPM



5.3 SPEED REGULATION AT VARIOUS LOAD CONDITIONS

Set RPM (RPM)	15Ω		40Ω		75Ω	
	Slot Sensor (RPM)	Tachometer (RPM)	Slot Sensor (RPM)	Tachometer (RPM)	Slot Sensor (RPM)	Tachometer (RPM)
800	794 – 805	796 – 802	797 – 808	798 – 805	799 – 807	800 – 808
900	896 – 905	893 – 902	897 – 907	897 – 905	898 – 910	899 – 906
1000	992 – 1004	993 – 1002	994 – 1006	996 – 1004	996 – 1008	997 – 1005
1100	1096 – 1105	1093 – 1103	1093 – 1107	1097 – 1104	1097 – 1111	1099 – 1106
1200	1194 – 1204	1194 – 1202	1192 – 1207	1194 – 1205	1198 – 1208	1199 – 1206
1300	1290 – 1304	1293 – 1302	1294 – 1306	1297 – 1305	1298 – 1306	1299 – 1307

5.4 EFFECT OF PID CONTROL ON MOTOR SPEED ACROSS VARYING LOAD WHEN TARGET SPEED IS SET T0 1000 RPM



CHAPTER 6: CONCLUSION

- Developed a PID based closed-loop DC motor control system using the PIC32MK MCM Curiosity Pro board.
- Implemented PWM-based speed control with real-time feedback from a slot sensor.
- Ensured safe operation with opto-isolation and MOSFET switching
- Achieved stable motor operation with good accuracy in tracking speed under varying loads.

CHAPTER 7: FUTURE WORK

- Extend the system to multi-axis telescope control
- Integrate an H-Bridge to enable bidirectional motor movement
- Add an encoder for accurate position and velocity control
- Replace the DC load with a mechanical load (e.g., calibrated loading or braking mechanism)
- Implement ramping profiles for smoother motor startup and control

CHAPTER 8: REFERENCES

- MICROCHIP TECHNOLOGY INC., PIC32MK1024MCM100 DATA SHEET, DS60001402D, 2020.[ONLINE AVAILABLE:HTTPS://WWW.MICROCHIP.COM]

- MICROCHIP TECHNOLOGY INC., PIC32 FAMILY REFERENCE MANUAL – SECTION 22: 12-BIT SAR ADC, 2020.[ONLINE AVAILABLE: HTTPS://WWW.MICROCHIP.COM]

- OMRON CORPORATION, EE-SX95 SERIES DATASHEET – PHOTOMICROSENSOR, CAT. NO. E435-E1-01, MAY 2013.[ONLINE] AVAILABLE: HTTPS://WWW.OMRON.COM

- VISHAY SEMICONDUCTORS, IRFZ44N N-CHANNEL MOSFET DATASHEET. [ONLINE AVAILABLE: HTTPS://WWW.VISHAY.COM]
- LITE-ON INC., 4N35 OPTOCOUPLER DATASHEET.[ONLINE AVAILABLE: HTTPS://WWW.LITEON.COM]